

UNIVERSIDAD SERGIO ARBOLEDA

PARCIAL 2: ALGORITMOS EVOLUTIVOS, GENÉTICOS, ASCENSO DE COLINAS, TEMPLE SIMULADO, ACO Y PSO

INTELIGENCIA ARTIFICIAL

DOCENTE
JOAQUÍN FERNANDO SÁNCHEZ CIFUENTES

INTEGRANTES
GABRIELA DELGADO TIBAMOSO
JULIAN CAMILO CARO HERRERA
NICOLÁS STEVEN PÉREZ CHUSCANO

2026

Punto 1. Algoritmos evolutivos y algoritmos genéticos

Se maximiza $f(x) = x^2$ con $x \in \{0, 1, \ldots, 31\}$, representando cada solución con un cromosoma binario de cinco bits. El máximo posible es `11111`, es decir, $x = 31$ y $f = 961$.

Parte A. Componente matemático y conceptual

1 y 2. Decodificación, aptitud y probabilidad de selección por ruleta

Cada cromosoma se lee como un número binario y su aptitud es el cuadrado de ese número. La probabilidad de selección es la aptitud del individuo dividida por la suma de aptitudes:

$$p_i = \frac{f(x_i)}{\sum_j f(x_j)}, \qquad \sum_j f(x_j) = 25 + 144 + 324 + 625 = 1118. \tag{1}$$

Tabla 1
Decodificación, aptitud y probabilidad de selección de la población inicial

Cromosoma	x	$f(x) = x^2$	p_i
`00101`	5	25	0,0224
`01100`	12	144	0,1288
`10010`	18	324	0,2898
`11001`	25	625	0,5590
Total		1118	1,0000

3. Cruce de un punto después del segundo bit

Se intercambian las colas de los padres `01|100` y `11|001`. Los descendientes son `01001` ($x = 9$, $f = 81$) y `11100` ($x = 28$, $f = 784$).

4. Mutación del último bit del primer descendiente

Al invertir el último bit de `01001` se obtiene `01000`: $x = 8$ y $f = 64$.

5. Preguntas conceptuales

Relación entre algoritmo evolutivo y algoritmo genético. Un algoritmo genético es un tipo de algoritmo evolutivo. Ambos mantienen una población de soluciones, la evalúan con una función

de aptitud y producen nuevas generaciones; el genético se distingue por representar las soluciones como cromosomas y usar cruce y mutación.

Función de selección, cruce y mutación. La selección da más probabilidad de reproducirse a los individuos más aptos, sin excluir a los demás. El cruce combina información de dos padres. La mutación introduce cambios aleatorios que ningún padre tenía.

Utilidad de una mutación que empeora la aptitud. La mutación mantiene la diversidad: puede crear un bit que no existe en la población y que es necesario para salir de un óptimo local, aunque en el momento produzca un individuo peor.

Parte B. Implementación en Python

Implementación

El algoritmo usa la población inicial dada, selección por ruleta, cruce de un punto con probabilidad 0,8, mutación independiente por bit con probabilidad 0,05, población constante de cuatro individuos, veinte generaciones y semilla 42 (módulo `random` de Python). El enunciado no define tres detalles, que se resolvieron así:

1. El punto de corte se elige al azar entre 1 y 4, porque un corte en 0 o en 5 produciría hijos idénticos a los padres. Si no hay cruce, los hijos son copias de los padres y luego pasan por la mutación.
2. Elitismo: se forman dos parejas y se obtienen cuatro hijos; el peor hijo se reemplaza por el mejor individuo de la generación anterior, que no se muta. Así la población se mantiene en cuatro y la mejor aptitud no puede disminuir.
3. Por cada pareja, los números aleatorios se piden en este orden: ruleta del primer padre, ruleta del segundo, decisión de cruce, punto de corte (solo si hay cruce) y mutación de los cinco bits de cada hijo.

Resultados

El mejor cromosoma final es `11111`, con $x = 31$ y aptitud 961. Se alcanzó en la generación 8 y se conservó hasta la 20. La mejor aptitud pasó de 625 a 841, 900 y 961, y nunca disminuyó (Tabla 2 y

Figura 1).

Tabla 2
Mejor aptitud y población por generación del algoritmo genético

Generación	Mejor aptitud	Población
0	625	`00101 01100 10010 11001`
1	841	`11100 11001 11101 11001`
2	841	`11101 11100 11100 11100`
3	841	`11101 11100 11100 11100`
4	900	`11101 11100 11101 11110`
5	900	`11110 11110 11100 11101`
6	900	`11110 11101 11110 11100`
7	900	`11101 11110 11110 11110`
8	961	`11111 11110 11111 11110`
9	961	`11111 11111 11111 11111`
10	961	`11011 11111 11111 11111`
11	961	`11011 11011 11111 11111`
12	961	`11111 11111 11110 11111`
13	961	`10111 11111 11111 11111`
14	961	`10111 11111 11111 11111`
15	961	`11111 11111 11111 11111`
16	961	`11111 11111 11111 11111`
17	961	`11111 11111 11111 11111`
18	961	`11111 11111 11111 11111`
19	961	`11111 11111 11111 11111`
20	961	`11111 11111 11111 11111`

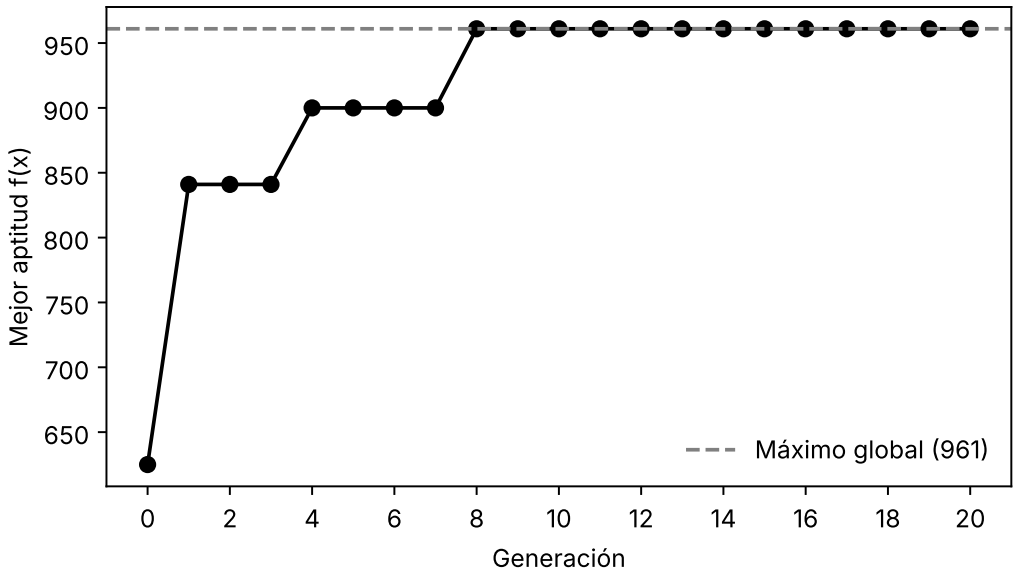
Nota. La generación 0 es la población inicial dada en el enunciado.

Pregunta de interpretación

El algoritmo sí encontró el óptimo global: 961 es el máximo de x^2 en $\{0,\ldots,31\}$. Sin embargo, veinte generaciones no garantizan encontrarlo. El algoritmo es estocástico, así que con otra semilla seguiría otra trayectoria. Con solo cuatro individuos la población pierde diversidad rápidamente, y si todos los cromosomas comparten un 0 en alguna posición, solo la mutación (probabilidad de 0,05 por bit) puede producir el 1 que falta; eso podría tardar más de veinte generaciones.

Figura 1

Historial de la mejor aptitud del algoritmo genético



Nota. La línea discontinua marca el máximo global, $f(31) = 961$.

Punto 2. Ascenso de colinas y óptimos locales

Se maximiza $f(x)$ en $\{0,\ldots,8\}$ con $f = (1,4,7,5,3,6,9,8,5)$. Los vecinos de x son $x-1$ y $x+1$ cuando están en el dominio; el algoritmo elige el vecino de mayor valor y solo se mueve si hay una mejora estricta.

Parte A. Componente matemático y conceptual

1. Ejecución desde $x_0 = 0$

Estados visitados: $0 \to 1 \to 2$. En $x = 2$ los vecinos valen $f(1) = 4$ y $f(3) = 5$; ninguno supera $f(2) = 7$ y el algoritmo se detiene en $x = 2$.

2. Ejecución desde $x_0 = 4$

Estados visitados: $4 \to 5 \to 6$. Desde 4 el mejor vecino es 5 (vale 6, contra 5 de $x = 3$). En $x = 6$ los vecinos valen 6 y 8, menos que $f(6) = 9$, y el algoritmo se detiene en $x = 6$.

3. Máximos locales y máximo global

Los máximos locales son $x = 2$ ($f = 7$) y $x = 6$ ($f = 9$). El máximo global es $x = 6$.

4. Influencia del estado inicial

El algoritmo solo acepta mejoras, así que nunca baja. El valle en $x = 4$ ($f = 3$) separa dos colinas, y el lado del valle donde empieza la búsqueda determina el pico al que llega.

5. Estrategia para evitar el máximo local

Ascenso de colinas con reinicios aleatorios: ejecutar el algoritmo desde varios estados iniciales y conservar el mejor resultado. El costo adicional es proporcional al número de reinicios: con k reinicios se hacen aproximadamente k veces más evaluaciones de f.

Parte B. Implementación en Python

Implementación

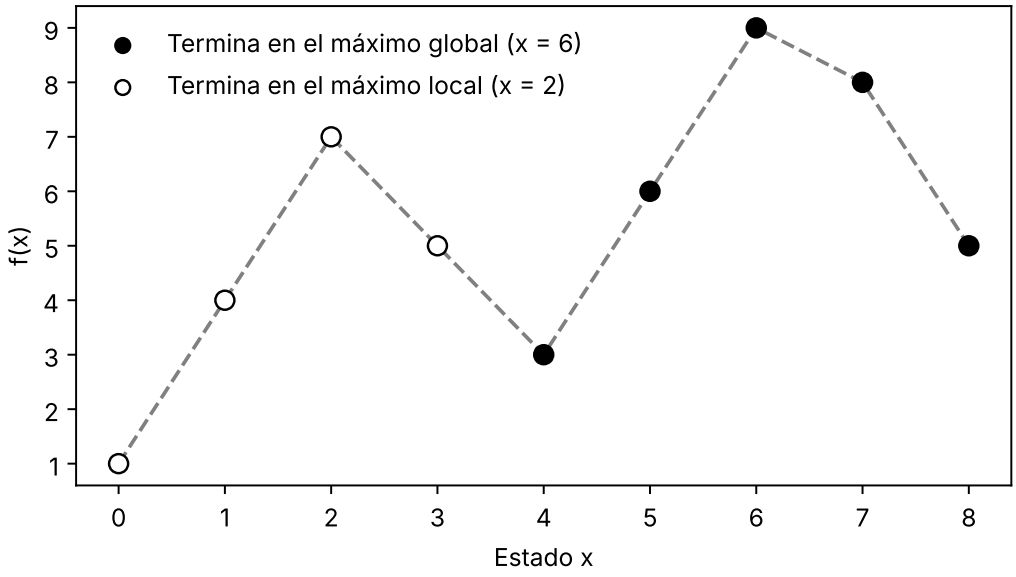
La función `ascenso_colinas(valores, inicio)` devuelve el estado final y la trayectoria. En cada paso calcula los vecinos válidos (en los bordes, $x = 0$ solo tiene el vecino 1 y $x = 8$ solo el 7), elige el de mayor valor y se detiene si ese valor no supera estrictamente al actual. La desigualdad estricta garantiza que el algoritmo termine: si aceptara vecinos de igual valor, en una meseta podría oscilar indefinidamente. Este punto no usa números aleatorios.

Resultados

Tabla 3
Ascenso de colinas desde cada estado inicial

Estado inicial	Trayectoria	Estado final	Valor final
0	$0 \to 1 \to 2$	2	7
1	$1 \to 2$	2	7
2	2	2	7
3	$3 \to 2$	2	7
4	$4 \to 5 \to 6$	6	9
5	$5 \to 6$	6	9
6	6	6	9
7	$7 \to 6$	6	9
8	$8 \to 7 \to 6$	6	9

Figura 2
Paisaje de búsqueda y estado final alcanzado desde cada estado inicial



Nota. Los puntos negros terminan en el máximo global ($x = 6$); los blancos, en el máximo local ($x = 2$).

Pregunta de interpretación

Cinco de los nueve estados iniciales (55,6 %), los estados 4 a 8, conducen al máximo global; los estados 0 a 3 terminan en el máximo local $x = 2$. Si el inicio se elige al azar, la probabilidad de fallar es $4/9$, y con k reinicios independientes la probabilidad de fallar en todos es $(4/9)^k$. Con tres reinicios es aproximadamente 0,088, de modo que el éxito sube a cerca de 91 %. Los reinicios aumentan la probabilidad de hallar el máximo global, pero no la garantizan.

Punto 3. Temple simulado: aceptación de soluciones peores

Se minimiza $E(x) = x^2$ con $x \in \{-5, \ldots, 5\}$ usando la regla de aceptación

$$P(\text{aceptar}) = \begin{cases} 1, & \Delta E \leq 0, \\ e^{-\Delta E/T}, & \Delta E > 0, \end{cases} \qquad \Delta E = E(x_{\text{nuevo}}) - E(x_{\text{actual}}), \tag{2}$$

donde una solución peor se acepta cuando $u < P(\text{aceptar})$.

Parte A. Componente matemático y conceptual

1. Cálculo de ΔE, probabilidad y decisión

Tabla 4
Decisión de aceptación en los tres casos del enunciado

Caso	Actual	Candidato	T	ΔE	$P(\text{aceptar})$	u	Decisión
A	3	2	5	$4-9=-5$	1	0,70	Acepta
B	2	3	5	$9-4=+5$	$e^{-1}=0{,}3679$	0,30	Acepta
C	2	3	1	$9-4=+5$	$e^{-5}=0{,}0067$	0,30	Rechaza

Nota. En el caso A el candidato mejora la energía y se acepta sin consultar u.

2. Influencia de la temperatura

Los casos B y C son el mismo movimiento; solo cambia la temperatura. Con $T = 5$ el empeoramiento se acepta con probabilidad 0,37 y con $T = 1$ con probabilidad 0,007. Una temperatura alta hace que el algoritmo acepte soluciones peores con frecuencia y explore; una temperatura baja hace que casi

solo acepte mejoras.

3. Enfriamiento demasiado rápido

La temperatura baja antes de que el algoritmo explore lo suficiente. La búsqueda se congela pronto y puede quedar atrapada en un óptimo local.

4. Comparación con el ascenso de colinas

El ascenso de colinas evalúa los vecinos, elige el mejor y nunca acepta un empeoramiento. El temple simulado elige un vecino al azar y puede aceptar un empeoramiento, con una probabilidad que disminuye a medida que baja la temperatura.

Parte B. Implementación en Python

Implementación

Se usó $x_0 = 5$, $T_0 = 10$, enfriamiento geométrico $T_{k+1} = 0{,}90\,T_k$, vecino aleatorio entre $x-1$ y $x+1$ (los válidos), cien iteraciones y semilla 42 (módulo `random`). En cada iteración se elige el candidato, se decide si se acepta, se actualiza la mejor solución si mejora y, al final, se enfría una vez; así la primera iteración usa $T = 10$. El número u solo se genera cuando $\Delta E > 0$. El estado actual y la mejor solución se guardan en variables distintas.

Resultados

La mejor solución es $x = 0$, con energía mínima $E = 0$, alcanzada en la iteración 9. Se aceptaron siete movimientos peores, todos con temperaturas entre 8,1 y 1,35 (Tabla 5). Desde la iteración 27 el estado no vuelve a cambiar, porque la temperatura es tan baja que no se acepta ningún empeoramiento. La Figura 3 muestra la temperatura y la energía, y la Tabla 6 el historial completo.

Tabla 5

Movimientos peores aceptados durante el temple simulado

Iteración	T	Movimiento	ΔE	$P(\text{aceptar})$
3	8,1000	$3 \to 4$	+7	0,4214
7	5,3144	$1 \to 2$	+3	0,5686
10	3,8742	$0 \to -1$	+1	0,7725
13	2,8243	$0 \to -1$	+1	0,7018
17	1,8530	$0 \to 1$	+1	0,5829
19	1,5009	$0 \to 1$	+1	0,5136
20	1,3509	$1 \to 2$	+3	0,1085

Nota. En cada uno de estos casos el número aleatorio u fue menor que $P(\text{aceptar})$.

Figura 3

Temperatura y energía por iteración del temple simulado

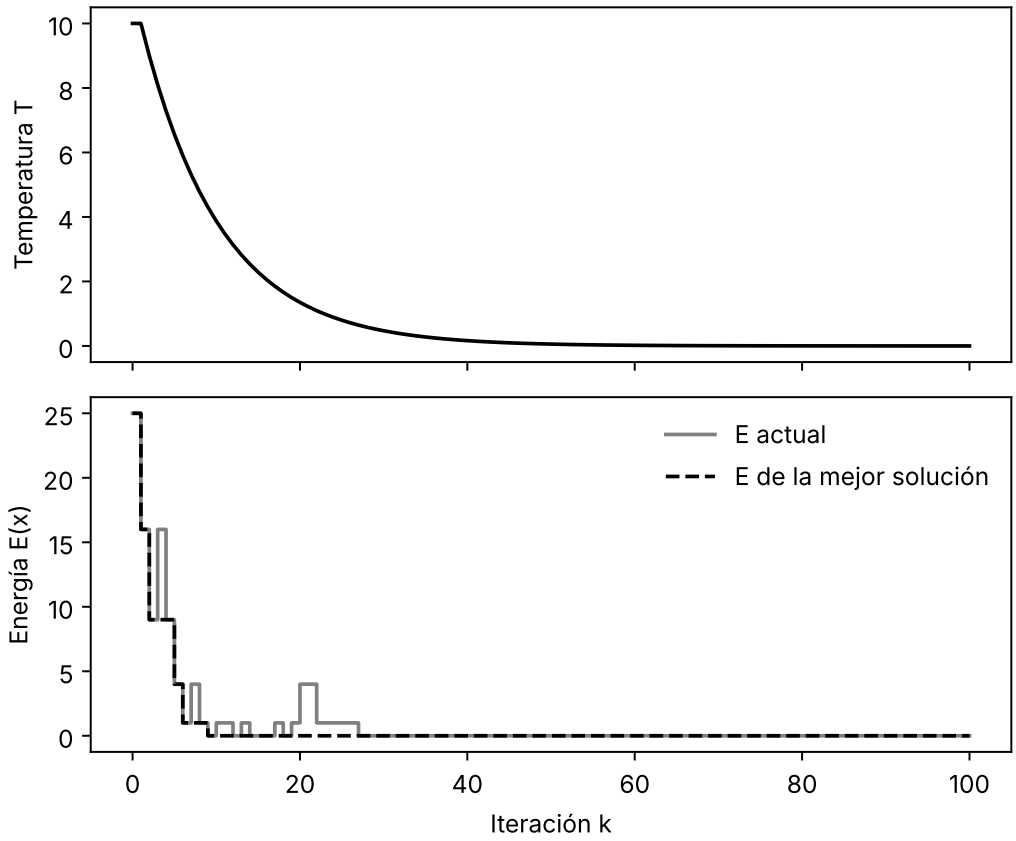


Tabla 6

Historial completo de temperatura y energía del temple simulado

k	T	x	$E(x)$	Mejor x	E mejor	k	T	x	$E(x)$	Mejor x	E mejor
0	10,0000	5	25	5	25	51	0,0515	0	0	0	0
1	10,0000	4	16	4	16	52	0,0464	0	0	0	0
2	9,0000	3	9	3	9	53	0,0417	0	0	0	0
3	8,1000	4	16	3	9	54	0,0376	0	0	0	0
4	7,2900	3	9	3	9	55	0,0338	0	0	0	0
5	6,5610	2	4	2	4	56	0,0304	0	0	0	0
6	5,9049	1	1	1	1	57	0,0274	0	0	0	0
7	5,3144	2	4	1	1	58	0,0246	0	0	0	0
8	4,7830	1	1	1	1	59	0,0222	0	0	0	0
9	4,3047	0	0	0	0	60	0,0200	0	0	0	0
10	3,8742	-1	1	0	0	61	0,0180	0	0	0	0
11	3,4868	-1	1	0	0	62	0,0162	0	0	0	0
12	3,1381	0	0	0	0	63	0,0146	0	0	0	0
13	2,8243	-1	1	0	0	64	0,0131	0	0	0	0
14	2,5419	0	0	0	0	65	0,0118	0	0	0	0
15	2,2877	0	0	0	0	66	0,0106	0	0	0	0
16	2,0589	0	0	0	0	67	0,0095	0	0	0	0
17	1,8530	1	1	0	0	68	0,0086	0	0	0	0
18	1,6677	0	0	0	0	69	0,0077	0	0	0	0
19	1,5009	1	1	0	0	70	0,0070	0	0	0	0
20	1,3509	2	4	0	0	71	0,0063	0	0	0	0
21	1,2158	2	4	0	0	72	0,0056	0	0	0	0
22	1,0942	1	1	0	0	73	0,0051	0	0	0	0
23	0,9848	1	1	0	0	74	0,0046	0	0	0	0
24	0,8863	1	1	0	0	75	0,0041	0	0	0	0
25	0,7977	1	1	0	0	76	0,0037	0	0	0	0
26	0,7179	1	1	0	0	77	0,0033	0	0	0	0
27	0,6461	0	0	0	0	78	0,0030	0	0	0	0
28	0,5815	0	0	0	0	79	0,0027	0	0	0	0
29	0,5233	0	0	0	0	80	0,0024	0	0	0	0
30	0,4710	0	0	0	0	81	0,0022	0	0	0	0
31	0,4239	0	0	0	0	82	0,0020	0	0	0	0
32	0,3815	0	0	0	0	83	0,0018	0	0	0	0
33	0,3434	0	0	0	0	84	0,0016	0	0	0	0
34	0,3090	0	0	0	0	85	0,0014	0	0	0	0
35	0,2781	0	0	0	0	86	0,0013	0	0	0	0
36	0,2503	0	0	0	0	87	0,0012	0	0	0	0
37	0,2253	0	0	0	0	88	0,0010	0	0	0	0
38	0,2028	0	0	0	0	89	0,0009	0	0	0	0
39	0,1825	0	0	0	0	90	0,0008	0	0	0	0
40	0,1642	0	0	0	0	91	0,0008	0	0	0	0
41	0,1478	0	0	0	0	92	0,0007	0	0	0	0
42	0,1330	0	0	0	0	93	0,0006	0	0	0	0
43	0,1197	0	0	0	0	94	0,0006	0	0	0	0
44	0,1078	0	0	0	0	95	0,0005	0	0	0	0
45	0,0970	0	0	0	0	96	0,0004	0	0	0	0
46	0,0873	0	0	0	0	97	0,0004	0	0	0	0
47	0,0786	0	0	0	0	98	0,0004	0	0	0	0
48	0,0707	0	0	0	0	99	0,0003	0	0	0	0
49	0,0636	0	0	0	0	100	0,0003	0	0	0	0
50	0,0573	0	0	0	0						

Nota. k es la iteración; la fila $k = 0$ es el estado inicial. T es la temperatura usada en esa iteración; el estado actual x y la mejor solución se registran después de la decisión de aceptación.

Pregunta de interpretación

Conviene guardar la mejor solución además del estado actual porque el estado actual puede empeorar, que es justamente lo que distingue al temple simulado. En esta ejecución el estado volvió a $x = 2$ en la iteración 20, después de haber estado en $x = 0$. Si el algoritmo terminara justo después de aceptar un empeoramiento y solo devolviera el estado actual, perdería el mejor punto visitado.

Punto 4. ACO: elección de rutas y actualización de feromonas

Parte A. Componente matemático y conceptual

1. Peso y probabilidad de cada alternativa

Con $\eta_{Aj} = 1/d_{Aj}$, $\alpha = 1$ y $\beta = 2$, el peso de cada arista es $\tau_{Aj}\,(1/d_{Aj})^2$ y su probabilidad es

$$p_{Aj} = \frac{\tau_{Aj}^{\alpha}\,\eta_{Aj}^{\beta}}{\sum_{k\in\{B,C,D\}} \tau_{Ak}^{\alpha}\,\eta_{Ak}^{\beta}}. \tag{3}$$

Tabla 7
Peso y probabilidad de elección desde el nodo A

Arista	d_{Aj}	τ_{Aj}	η_{Aj}	Peso $\tau\,\eta^2$	p_{Aj}
A→B	2	1	0,50	0,250	0,6024
A→C	4	2	0,25	0,125	0,3012
A→D	5	1	0,20	0,040	0,0964
Suma				0,415	1,0000

2. Alternativa más probable

La más probable es A→B, con 60,24 %. Su elección no está garantizada: la hormiga elige de forma probabilística, por ruleta, y queda una probabilidad de 39,76 % de que elija A→C o A→D. Esa aleatoriedad permite explorar otras rutas.

3. Actualización de feromonas

La hormiga completó una ruta de longitud $L = 10$ que usa A→B, de modo que $\Delta\tau_{AB} = Q/L = 10/10 = 1$ y el depósito en las otras aristas es cero. Con $\rho = 0{,}2$:

$$\tau_{AB} = 0{,}8 \cdot 1 + 1 = 1{,}8, \qquad \tau_{AC} = 0{,}8 \cdot 2 + 0 = 1{,}6, \qquad \tau_{AD} = 0{,}8 \cdot 1 + 0 = 0{,}8. \tag{4}$$

4. Función de la evaporación

La evaporación reduce gradualmente la feromona de todas las aristas. Así el sistema olvida decisiones antiguas o malas, la feromona no crece sin límite y las rutas que nadie refuerza pierden atractivo. Si la feromona dominara excesivamente la elección, casi todas las hormigas seguirían el mismo camino, se perdería la exploración y el algoritmo podría converger prematuramente a una ruta que parece buena pero no es la mejor.

Parte B. Implementación en Python

Implementación

Cada ruta completa de A a Z tiene una sola feromona, inicialmente 1, y su probabilidad es $p_r = \tau_r^{\alpha}(1/L_r)^{\beta} / \sum_s \tau_s^{\alpha}(1/L_s)^{\beta}$. Se usaron diez hormigas por iteración, veinte iteraciones, $\alpha = 1$, $\beta = 2$, $\rho = 0{,}2$, $Q = 10$ y semilla 42 (módulo `random`). Cada iteración sigue los cuatro pasos del enunciado en orden: probabilidades con las feromonas actuales, elección de las diez hormigas por ruleta con esas mismas probabilidades, evaporación de todas las rutas y depósito de Q/L_r por cada hormiga en la ruta que eligió. La iteración 1 se puede comprobar a mano: con 6, 3 y 1 elecciones, $\tau_{ABZ} = 0{,}8 + 6 \cdot 10/6 = 10{,}8$, $\tau_{ACZ} = 0{,}8 + 3 \cdot 10/8 = 4{,}55$ y $\tau_{ADZ} = 0{,}8 + 10/10 = 1{,}8$.

Resultados

Frecuencias de elección: de las 200 elecciones, A-B-Z ($L = 6$) se eligió 187 veces (93,5 %), A-C-Z ($L = 8$) 12 veces (6,0 %) y A-D-Z ($L = 10$) una vez (0,5 %).

Feromonas finales: 79,2315 en A-B-Z, 2,3580 en A-C-Z y 0,0259 en A-D-Z.

Mejor ruta observada: A-B-Z, que es a la vez la ruta más corta elegida por alguna hormiga

y la más elegida. La Tabla 8 muestra el detalle por iteración y la Figura 4, la evolución.

Tabla 8
Probabilidades, elecciones y feromonas por iteración del modelo ACO

	Probabilidad			Elecciones			Feromona al final		
Iter.	ABZ	ACZ	ADZ	ABZ	ACZ	ADZ	ABZ	ACZ	ADZ
1	0,5202	0,2926	0,1873	6	3	1	10,8000	4,5500	1,8000
2	0,7710	0,1827	0,0463	9	1	0	23,6400	4,8900	1,4400
3	0,8785	0,1022	0,0193	9	1	0	33,9120	5,1620	1,1520
4	0,9109	0,0780	0,0111	9	1	0	42,1296	5,3796	0,9216
5	0,9262	0,0665	0,0073	10	0	0	50,3703	4,3037	0,7373
6	0,9494	0,0456	0,0050	10	0	0	56,9629	3,4429	0,5898
7	0,9636	0,0328	0,0036	9	1	0	60,5704	4,0044	0,4719
8	0,9615	0,0358	0,0027	10	0	0	65,1230	3,2035	0,3775
9	0,9711	0,0269	0,0020	9	1	0	67,0984	3,8128	0,3020
10	0,9675	0,0309	0,0016	8	2	0	67,0120	5,5502	0,2416
11	0,9543	0,0445	0,0012	10	0	0	70,2763	4,4402	0,1933
12	0,9648	0,0343	0,0010	10	0	0	72,8877	3,5521	0,1546
13	0,9726	0,0267	0,0007	10	0	0	74,9768	2,8417	0,1237
14	0,9786	0,0209	0,0006	10	0	0	76,6481	2,2734	0,0990
15	0,9831	0,0164	0,0005	10	0	0	77,9852	1,8187	0,0792
16	0,9867	0,0129	0,0004	9	1	0	77,3881	2,7050	0,0633
17	0,9804	0,0193	0,0003	10	0	0	78,5772	2,1640	0,0507
18	0,9845	0,0153	0,0002	10	0	0	79,5284	1,7312	0,0405
19	0,9877	0,0121	0,0002	10	0	0	80,2894	1,3849	0,0324
20	0,9902	0,0096	0,0001	9	1	0	79,2315	2,3580	0,0259

Nota. Las probabilidades se calculan con la feromona al inicio de la iteración; la feromona al final incluye la evaporación y los depósitos de esa iteración.

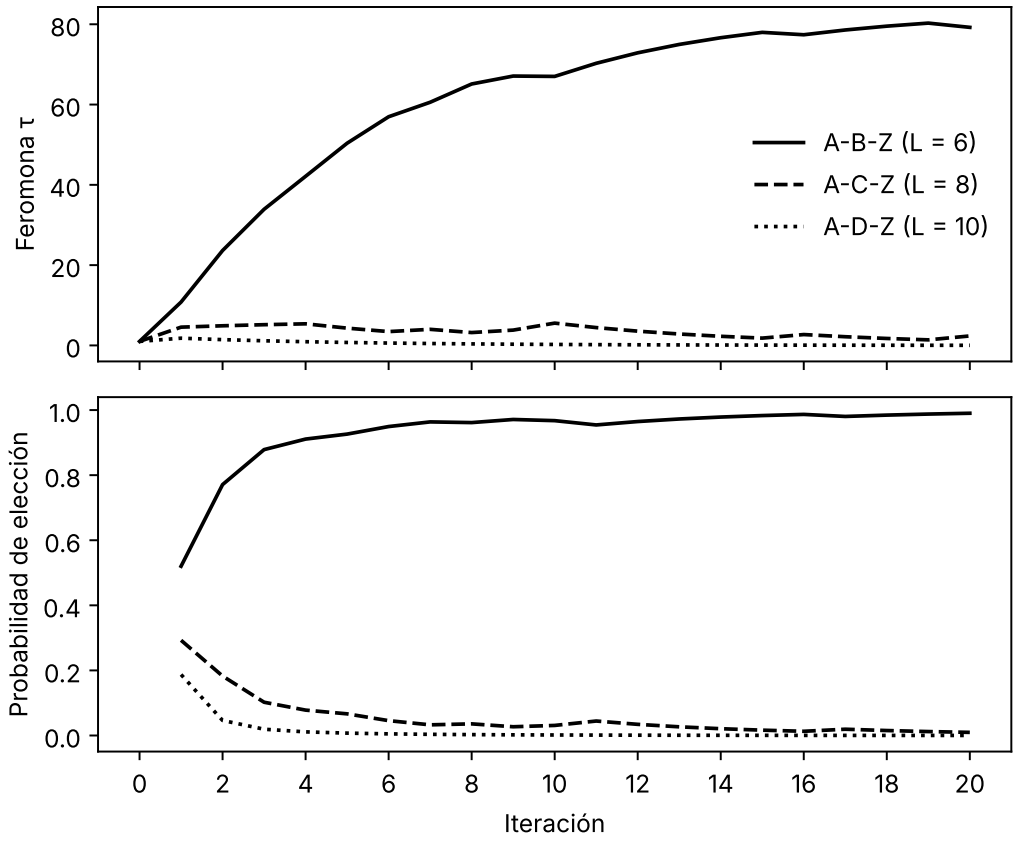
Pregunta de interpretación

La calidad de las rutas influye de dos formas. La ruta más corta tiene mayor información heurística $(1/L)^2$, así que desde el inicio es la más probable (52 % en la iteración 1). Además, cada hormiga que la elige deposita más feromona ($Q/6 \approx 1{,}67$, frente a $Q/8 = 1{,}25$ y $Q/10 = 1{,}0$), por lo que su feromona crece más rápido.

La experiencia colectiva funciona como una memoria compartida: cada hormiga deja feromona en la ruta que elige y las siguientes la tienen en cuenta. Esto produce una retroalimentación positiva: la ruta buena atrae más hormigas, que depositan más feromona y la hacen todavía más

Figura 4

Feromona y probabilidad de elección de cada ruta por iteración



atractiva. La probabilidad de A-B-Z pasó de 0,52 en la iteración 1 a 0,99 en la iteración 20, mientras la evaporación redujo la feromona de las rutas poco usadas (A-D-Z terminó en 0,026). Aun así, A-C-Z se siguió eligiendo ocasionalmente, porque la elección es probabilística.

Punto 5. PSO: movimiento de partículas y memoria

Parte A. Componente matemático y conceptual

1. Términos de inercia, cognitivo y social

Con $x = 4$, $v = -1$, $p_{\text{best}} = 2$, $g_{\text{best}} = 1$, $w = 0{,}5$, $c_1 = c_2 = 1{,}5$, $r_1 = 0{,}4$ y $r_2 = 0{,}6$:

$$\text{Inercia:} \quad wv = 0{,}5 \cdot (-1) = -0{,}5, \tag{5}$$

$$\text{Cognitivo:} \quad c_1 r_1 (p_{\text{best}} - x) = 1{,}5 \cdot 0{,}4 \cdot (2-4) = -1{,}2, \tag{6}$$

$$\text{Social:} \quad c_2 r_2 (g_{\text{best}} - x) = 1{,}5 \cdot 0{,}6 \cdot (1-4) = -2{,}7. \tag{7}$$

2. Nueva velocidad, posición y aptitud

$v_{\text{nuevo}} = -0{,}5 - 1{,}2 - 2{,}7 = -4{,}4$; $x_{\text{nuevo}} = 4 + (-4{,}4) = -0{,}4$; $f(-0{,}4) = 0{,}16$.

3. Actualización de p_{best} y g_{best}

Como $f(-0{,}4) = 0{,}16 < f(2) = 4$, se actualiza $p_{\text{best}} = -0{,}4$. Como también $0{,}16 < f(1) = 1$ y las demás partículas no mejoraron, se actualiza $g_{\text{best}} = -0{,}4$.

4. Mejor posición personal frente a mejor posición global

p_{best} es la mejor posición que ha visitado una partícula en particular: su memoria individual. Alimenta el término cognitivo, que mantiene la diversidad del enjambre. g_{best} es la mejor posición encontrada por todo el enjambre: la memoria colectiva. Alimenta el término social, que guía hacia la mejor zona conocida y favorece la convergencia.

5. Intercambio de información en PSO y en ACO

En PSO la información se comparte de forma directa y explícita: todas las partículas conocen una misma posición, g_{best}, que es un punto del espacio continuo. En ACO la comunicación es indirecta (estigmergia): las hormigas depositan feromona en las aristas y las siguientes la perciben. En ambos casos una memoria colectiva sesga las decisiones futuras, pero en ACO esa memoria se olvida por evaporación, mientras que en PSO g_{best} solo se reemplaza cuando aparece una posición mejor.

Parte B. Implementación en Python

Implementación

Se minimiza $f(x,y) = x^2 + y^2$ en $[-5,5]^2$ con diez partículas, posiciones iniciales uniformes, velocidades iniciales en cero, $w = 0{,}5$, $c_1 = c_2 = 1{,}5$, treinta iteraciones y semilla 42, con el generador de NumPy `default_rng(42)`. Posiciones y velocidades son matrices de 10×2, y en cada iteración se generan matrices nuevas r_1 y r_2 de 10×2, de modo que los valores son independientes por partícula, dimensión e iteración. Todas las partículas se mueven con el mismo g_{best}; después se evalúan y se actualizan las memorias personales y la global, solo con mejoras estrictas. Si una coordenada sale del dominio, se ajusta al límite y esa componente de la velocidad se pone en cero.

Resultados

La mejor posición final es $(x,y) \approx (-1{,}848 \times 10^{-4};\ 1{,}388 \times 10^{-5})$ y el valor de la función es $f \approx 3{,}434 \times 10^{-8}$. La mejor aptitud global bajó de 7,7601 en el estado inicial a 0,6533 en la primera iteración y siguió bajando hasta la iteración 27 (Tabla 9 y Figura 5). El historial nunca aumenta, porque g_{best} solo se reemplaza cuando hay una mejora estricta.

Tabla 9

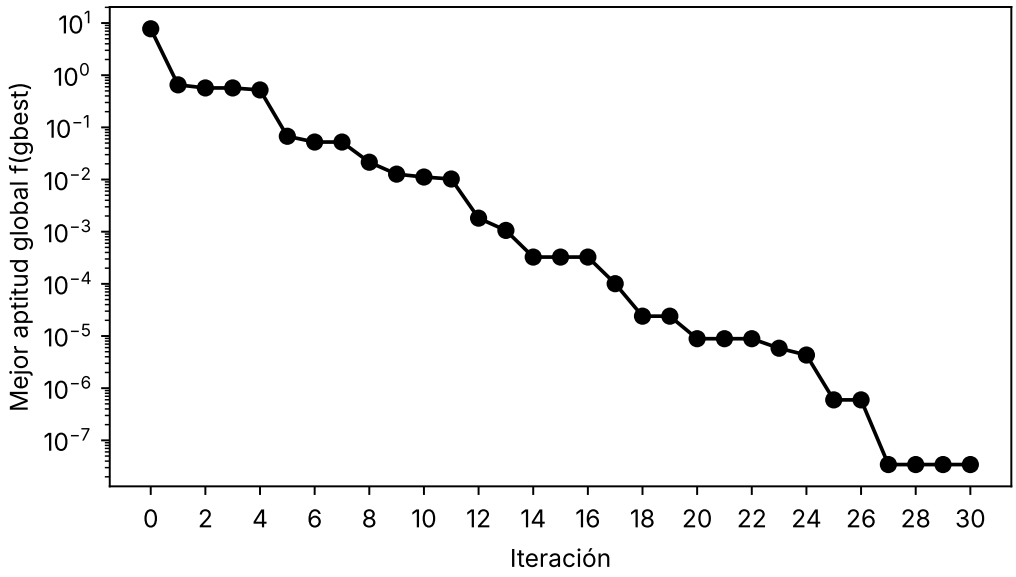
Historial de la mejor aptitud global del PSO

Iter.	$f(g_{\text{best}})$	Iter.	$f(g_{\text{best}})$	Iter.	$f(g_{\text{best}})$
0	7,7601	11	0,0102	22	$8{,}881 \times 10^{-6}$
1	0,6533	12	0,0018	23	$5{,}834 \times 10^{-6}$
2	0,5698	13	0,0011	24	$4{,}319 \times 10^{-6}$
3	0,5698	14	$3{,}262 \times 10^{-4}$	25	$5{,}965 \times 10^{-7}$
4	0,5216	15	$3{,}262 \times 10^{-4}$	26	$5{,}965 \times 10^{-7}$
5	0,0675	16	$3{,}262 \times 10^{-4}$	27	$3{,}434 \times 10^{-8}$
6	0,0523	17	$1{,}008 \times 10^{-4}$	28	$3{,}434 \times 10^{-8}$
7	0,0523	18	$2{,}402 \times 10^{-5}$	29	$3{,}434 \times 10^{-8}$
8	0,0215	19	$2{,}402 \times 10^{-5}$	30	$3{,}434 \times 10^{-8}$
9	0,0127	20	$8{,}881 \times 10^{-6}$		
10	0,0112	21	$8{,}881 \times 10^{-6}$		

Nota. La iteración 0 corresponde al enjambre inicial.

Figura 5

Historial de la mejor aptitud global del PSO



Nota. El eje vertical está en escala logarítmica porque la aptitud disminuye en ocho órdenes de magnitud.

Pregunta de interpretación

Las partículas pueden converger sin alcanzar exactamente $(0,0)$ por varias razones. El espacio es continuo, así que la probabilidad de caer exactamente en ese punto es prácticamente nula. Los factores r_1 y r_2 cambian en cada iteración, por lo que las partículas se acercan con pasos irregulares. Además, son atraídas hacia sus p_{best} y hacia g_{best}, no hacia el óptimo: si g_{best} está cerca de $(0,0)$ pero no en él, el enjambre se concentra alrededor de ese punto. Con $w = 0{,}5$ la velocidad se amortigua y las mejoras son cada vez más pequeñas, y con treinta iteraciones el error queda en el orden de 10^{-8}.